



We use optional cookies to improve your experience on our websites, such as through social media connections, and to display personalized advertising based on your online activity. If you reject optional cookies, only cookies necessary to provide you the services will be used. You may change your selection by clicking “Manage Cookies” at the bottom of the page.
[Privacy Statement](#)
[Third-Party Cookies](#)

Accept

Reject

Manage cookies

Getting Started

⚠ GraphRAG can consume a lot of LLM resources! We strongly recommend starting with the tutorial dataset here until you understand how the system works, and consider experimenting with fast/inexpensive models first before committing to a big indexing job.

Requirements

[Python 3.10-3.12](#)

The following is a simple end-to-end example for using GraphRAG on the command line after installing from [pypi](#).

It shows how to use the system to index some text, and then use the indexed data to answer questions about the documents.

Install GraphRAG

To get started, create a project space and python virtual environment to install `graphrag`.

Create Project Space

```
mkdir graphrag_quickstart
cd graphrag_quickstart
python -m venv .venv
```

Activate Python Virtual Environment - Unix/MacOS

```
source .venv/bin/activate
```

Activate Python Virtual Environment - Windows

```
.venv\Scripts\activate
```

Install GraphRAG

```
python -m pip install graphrag
```

Initialize GraphRAG

To initialize your workspace, first run the `graphrag init` command.

```
graphrag init
```

When prompted, specify the default chat and embedding models you would like to use in your config.

This will create two files, `.env` and `settings.yaml`, and a directory `input`, in the current directory.

- `input` Location of text files to process with `graphrag`.

- `.env` contains the environment variables required to run the GraphRAG pipeline. If you inspect the file, you'll see a single environment variable defined, `GRAPHRAG_API_KEY=<API_KEY>`. Replace `<API_KEY>` with your own OpenAI or Azure API key.
- `settings.yaml` contains the settings for the pipeline. You can modify this file to change the settings for the pipeline.

Download Sample Text

Get a copy of A Christmas Carol by Charles Dickens from a trusted source:

```
curl https://www.gutenberg.org/cache/epub/24022/pg24022.txt -o ./input/book.txt
```

Set Up Workspace Variables

Using OpenAI

If running in OpenAI mode, you only need to update the value of `GRAPHRAG_API_KEY` in the `.env` file with your OpenAI API key.

Using Azure OpenAI

In addition to setting your API key, Azure OpenAI users should set the variables below in the `settings.yaml` file. To find the appropriate sections, just search for the `models:` root configuration; you should see two sections, one for the default chat endpoint and one for the default embeddings endpoint. Here is an example of what to add to the chat model config:

```
type: chat
model_provider: azure
model: gpt-4.1
deployment_name: <AZURE_DEPLOYMENT_NAME>
api_base: https://<instance>.openai.azure.com
api_version: 2024-02-15-preview # You can customize this for other versions
```

Using Managed Auth on Azure

To use managed auth, edit the `auth_type` in your model config and *remove* the `api_key` line:

```
auth_type: azure_managed_identity # Default auth_type is is api_key
```

You will also need to login with [az login](#) and select the subscription with your endpoint.

Index

Now we're ready to index!

```
graphrag index
```

```
Loading csv files from ./input
Loading 1 csv files
Total number of unfiltered csv rows: 4748
Final # of rows loaded: 4748
! Executing Pipeline...
└─ Loading Input (csv) - 1 files loaded (0 filtered) 100% 0:00:00 0:00:00
└─ Workflow: create_base_text_units
└─ Workflow: create_base_extracted_entities
└─ verb: entity_extract 99% 0:01:20 12:22:38
```

This process will usually take a few minutes to run. Once the pipeline is complete, you should see a new folder called `./output` with a series of parquet files.

Query

Now let's ask some questions using this dataset.

Here is an example using Global search to ask a high-level question:

```
graphrag query "What are the top themes in this story?"
```

Here is an example using Local search to ask a more specific question about a particular character:

```
graphrag query \
  "Who is Scrooge and what are his main relationships?" \
  --method local
```

Please refer to [Query Engine](#) docs for detailed information about how to leverage our Local and Global search mechanisms for extracting meaningful insights from data after the Indexer has wrapped up execution.

Going Deeper

- For more details about configuring GraphRAG, see the [configuration documentation](#).
- To learn more about Initialization, refer to the [Initialization documentation](#).
- For more details about using the CLI, refer to the [CLI documentation](#).
- Check out our [visualization guide](#) for a more interactive experience in debugging and exploring the knowledge graph.